

Database Abstraction

The routes ask for an operation. The database module carries it out.

Gregory S. DeLozier, PhD

Why an Abstraction Layer?

A named operation hides the details of carrying it out.

```
pet = database.get_pet(id)
```

The route asks for a pet. The module handles the query.

Query details can change while the calling code stays the same.

Single Responsibility Principle

Single Responsibility Principle (SRP): one reason to change.

What changes?	Where it belongs
Request handling or page selection	<code>app.py</code>
How pets are stored or retrieved	<code>database.py</code>

Keep code that changes for the same reason together.

Running the Application

```
flask run
```

```
http://127.0.0.1:5000/list
```

Separate database setup: `python3 setup_database.py`

Two Modules, Different Work

app.py

Read the request

Call an operation

Render a template

Choose a redirect

database.py

Open the database

Execute SQL

Return records

Commit writes

The Database Interface

Function	Result
<code>get_pets()</code>	List of dictionaries
<code>get_pet(id)</code>	Dictionary or None
<code>create_pet(data)</code>	Saved new record
<code>update_pet(id, data)</code>	Saved changes
<code>delete_pet(id)</code>	Removed record

Preparing the Table

```
app = Flask(__name__)
database.setup_database("pets.db")
```

Pet fields

Identity	id, name
Description	type, age
Care	food, owner

Named Records

```
connection.row_factory = sqlite3.Row
```

```
pets = cursor.fetchall()  
pets = [dict(pet) for pet in pets]
```

`pet['food']` and `pet['owner']` identify fields directly.

Listing Pets

```
@app.route("/pets", methods=["GET"])
@app.route("/list", methods=["GET"])
def get_pets():
    return render_template(
        "list.html", pets=database.get_pets()
    )
```

The template chooses the displayed field order.

Creating a Pet

```
@app.route("/create", methods=["POST"])
def post_create():
    database.create_pet(dict(request.form))
    return redirect(url_for("get_pets"))
```

The database function commits before returning.

Inside create_pet

```
cursor.execute(  
    "INSERT INTO pet(name, age, type, food, owner) "  
    "VALUES (?, ?, ?, ?, ?)",  
    (data["name"], data["age"], data["type"],  
     data["food"], data["owner"]),  
)  
connection.commit()
```

The column list determines the parameter order.

Loading One Pet

```
cursor.execute("SELECT * FROM pet WHERE id = ?", (id,))  
pet = cursor.fetchone()  
if pet is None:  
    return None  
return dict(pet)
```

A dictionary means found. None means absent.

Filling the Edit Form

```
data = database.get_pet(id)
```

```
<input name="food" value="{{data['food'] or ''}}"/>  
<input name="owner" value="{{data['owner']}}"/>
```

The form submits to `/update/{{data['id']}}`.

Saving the Edit

```
database.update_pet(id, dict(request.form))  
return redirect(url_for("get_pets"))
```

```
UPDATE pet SET name=?, age=?, type=?, food=?, owner=?  
WHERE id=?
```

Five replacement values, followed by the identifier.

Deleting a Pet

```
@app.route("/delete/<int:id>", methods=["GET"])
def get_delete(id):
    database.delete_pet(id)
    return redirect(url_for("get_pets"))
```

The database function deletes and commits.

Testing the Same Functions

```
python3 database.py
```

```
delete_pet(pet["id"])  
assert get_pet(pet["id"]) is None
```

The tests use `test_pets.db`.

Following the Call

Route: What operation does this request ask for?

Database function: Which values and statement carry it out?

Response: How does the browser show the result?

Use the function boundary to follow one operation at a time.